

debounce를 직접 구현하며 배운 5가지

debounce를 직접 구현했습니다.

```
function debounce<T>(func: () => T, delay: number): () => T {  
  let timer: number | null = null;  
  let result: T;  
  
  return () => {  
    if (timer) clearTimeout(timer);  
    timer = setTimeout(() => {  
      result = func();  
      timer = null;  
    }, delay);  
    return result;  
  };  
}
```

핵심 동작은 잘 돌아갔습니다. 그런데 가장 잘 알려진 라이브러리 중 하나인 **es-toolkit**소스와 비교하며 다섯 가지의 개선점을 발견했습니다.

1. 제네릭과 Parameters Utility Types로 함수의 인자 타입 보존하기.
2. 화살표 함수의 `this` 는 어디서 오는가
3. TS의 `this` 파라미터는 컴파일하면 사라진다
4. `ReturnType<typeof setTimeout>` 을 쓰는 진짜 이유
5. 덕 타이핑, 그리고 TypeScript의 구조적 타이핑

① 제네릭으로 타입을 보존한다.

Before — 받는 순간 타입이 사라집니다.

```
function debounce(func: (...args: any[]) => void): (...args: any[]) => void
const save = debounce((id: number, text: string) => { ... }, 300);
save(true);    // 컴파일 에러가 나야하는데 통과된다.
```

After — 제네릭 타입에 담아 그대로 통과시킵니다:

```
function debounce<F extends (...args: any[]) => void>(
  func: F, ...
): DebouncedFunction<F>    // 내부: (...args: Parameters<F>) => void

save(true);    // 컴파일 에러 - [id: number, text: string] 튜플이 살아있다
```

- `F` — 호출 시점에 원본 함수 타입이 담기는 **타입 변수**
- `Parameters<F>` — 인자 튜플 추출. 이때 `this` 파라미터는 **암시적으로 제거**되므로, `this` 가 선언된 함수를 넘겨도 인자 타입이 그대로 통과합니다

Parameters는 parameter로 전달되는 `this` 를 제외합니다.

```
type Fn = (this: Window, a: number, b: string) ⇒ void;  
type P = Parameters<Fn>; // [a: number, b: string] - this 제외  
type T = ThisParameterType<Fn>; // Window - this는 별도 채널에 보관된다
```

② 화살표 함수의 this

"화살표 함수는 자신이 **정의된** 바깥 스코프의 this를 참조한다" (렉시컬 스코프)

```
const obj = {  
  name: 'kim',  
  arrow: () => console.log(this?.name),  
  normal() {  
    console.log(this.name);  
  },  
};
```

```
obj.arrow(); // undefined - 호출자는 obj지만, "정의된 곳"은 모듈 스코프  
obj.normal(); // 'kim' - 함수 표현식으로 된 만 호출 방식(obj.)이 this를 결정
```

호출자(caller)가 기준이라면 `obj.arrow()` 의 this는 `obj` 를 가리켜야 합니다.

일반 함수의 this = **호출 시점**에 결정 (동적) / 화살표의 this = **정의 위치**에 고정 (정적)
"누가 불렀나"가 아니라 **"어디서 태어났나"**

마찬가지로, debounce에서도 화살표 함수를 쓰면 this가 유실됩니다.

```
const counter = {
  count: 0,
  increment() {
    this.count += 1;
  },
};

counter.increment = debounce(counter.increment, 300);
counter.increment(); // 호출은 성공 → 300ms 뒤 TypeError: Cannot read properties of undefined
```

counter.increment() 로 불렀으니 **debounced 함수**의 this는 counter 가 호출되어야 합니다.
하지만 내부에서 func(... args) 로 그냥 호출하는 순간 ****원본 increment****의 this는 undefined 가 됩니다.

this는 호출자가 넘기는 **암묵적 인자**다.

따라서, func.apply(this, args) 로 전달하여 해당 문제를 해결할 수 있었습니다.

③ this 파라미터는 타입스크립트에만 존재한다

```
// TypeScript  
const debounced = function (this: any, ... args: Parameters<F>) { ... };
```

```
// 컴파일된 JavaScript - this 파라미터가 통째로 사라진다  
const debounced = function (... args) { ... };
```

- this는 반드시 첫 파라미터로 존재해야 합니다.
- 호출할 때 넘기는 값이 아니라, 암시적으로 넘겨지는 값 입니다.

주의 : 선언만으로는 아무것도 보존되지 않습니다.

```
timer = setTimeout(() => {  
  func.apply(this, args); // 화살표 콜백이 debounced의 this를 렉시컬 상속 → apply로 전달  
}, delay);
```


④ ReturnType<typeof setTimeout> 주체

브라우저는 timer의 ID를, Node는 `Timeout` 객체를 반환합니다.

그럼 타입은 누가 판단하나?

- 런타임 환경이 아니라, 컴파일 시점의 타입 선언에 따라 판단합니다.
- `typeof setTimeout` — 지금 스코프에 로드된 **선언을 조회** (lib.dom.d.ts vs @types/node)
- `ReturnType< ... >` — 그 선언에서 **반환 타입을 추출**

→ `number` 나 `NodeJS.Timeout` 을 하드코딩하지 않고 **선언에서 파생**시키므로, 어떤 환경에서 컴파일해도 알아서 맞는 타입이 됩니다.

⑤ 덕 타이핑 — 그리고 TS의 구조적 타이핑

"오리처럼 걷고, 오리처럼 꺾꺾거리면 — 그것은 오리다."

객체의 혈통(클래스, 상속 관계, 선언된 이름)이 아니라, 실제로 가진 **속성과 행동** 으로 타입을 판단하는 방식.

	판단 기준
덕 타이핑 (JS 런타임)	지금 그 메서드가 있는가
구조적 타이핑 (TS)	구조가 맞는가 — 컴파일 타임의 덕 타이핑
명목적 타이핑 (Java 등)	선언된 이름·상속 관계

감사합니다